Solving the Self-Similar Burgers Equation via Chebyshev Collocation and Gradient-Based Optimization

Allen Yang

December 6, 2025

Textbook Chapter: Chapter 10, Boundary Value Problems for ODEs;
Contribution of the Authors: Worked by myself;
Use of AI tools: I used AI tools to help with code development, generating ideas, and writing the final report;
Other Sources: I consulted with Prof. Koumoutsakos to discuss approaches for the problem of inferring the parameter λ.

1 Introduction

Finite time self-similar blowups are a fundamental phenomenon in the study of nonlinear partial differential equations, especially in the context of fluid dynamics and nonlinear evolution equations such as the Navier-Stokes equations. A *blowup* refers to the formation of a singularity in finite time, where the solution (or its derivatives) becomes unbounded as some critical time is approached.

In a *self-similar blowup*, the solution near the singularity exhibits a structure that is invariant under a specific scaling of space and time. That is, as the singularity is approached, the solution can often be represented in terms of a single similarity variable—a specific combination of space and time. This allows one to seek solutions of the PDE in a reduced form, often involving similarity variables, which convert the original problem into an ordinary differential equation (ODE) or a simpler PDE.

One of the simplest PDEs which exhibits self-similar blowups is the inviscid Burgers equation,

$$u_t + uu_x = 0, \tag{1}$$

in which smooth initial conditions can develop a singularity in finite time. Taking the local self-similar ansatz

$$u(x,t) = (1-t)^{\lambda} U(y(x,t)), \quad y(x,t) = \frac{x}{(1-t)^{1+\lambda}}, \tag{2}$$

the self-similar Burgers equation becomes

$$-\lambda U + ((1+\lambda)y + U)d_y U = 0, \tag{3}$$

which is a first order nonlinear ODE for $U(y)$. If we additionally impose U to be odd and add the boundary conditions $U(-2) = 1$ and $U(2) = -1$, then the analytical self-similar Burgers' solution is given (implictly) by:

$$y = -U - CU^{1+\frac{1}{\lambda}}. \tag{4}$$

Thus, for this project, we consider the boundary value problem

$$\begin{cases} -\lambda U + ((1+\lambda)y + U)d_y U = 0, \quad y \in (-2,2), \\ U(-2) = 1, \quad U(2) = -1. \end{cases} \tag{5}$$

To understand blowup in this context, notice that the spatial derivative is $\partial_x u = (1-t)^{-\lambda} d_y U$ under the self-similar ansatz. This means if $d_y U$ is nontrivial, then as $t \to 1^-$, the factor $(1-t)^{-\lambda}$ diverges for any $\lambda > 0$, leading to a blowup in the gradient of $u(x,t)$. At the initial time $t = 0$, the similarity variable reduces to $y(x,0) = x$, so if $U(y)$ is smooth within $y \in (-2,2)$, then $u(x,0)$ is smooth for $x \in (-2,2)$. As time advances towards $t = 1$, the region described by the fixed "inner region" $y \in [-2,2]$ corresponds to an increasingly small neighborhood around $x = 0$ in (x,t) space, capturing precisely where the gradient becomes unbounded. Thus, the primary objective is to find a smooth solution $U(y)$ to the boundary value problem (5), which allows us to clearly demonstrate how smooth initial conditions can evolve into a finite-time singularity. We focus on the inviscid Burgers equation due to its well-understood analytical properties, particularly the existence of an explicit self-similar solution (4), as well as the discrete set of parameter values λ admit smooth solutions to the boundary value problem, given by

$$\lambda = \frac{1}{2(j+1)}, \qquad j = 0, 1, 2, \ldots. \tag{6}$$

For other λ, the solutions demonstrate nonsmooth behavior at $y = 0$.

Related work done in [2] demonstrates the application of Physics-Informed Neural Networks (PINNs) to the self-similar Burgers equation problem. In their approach, they parameterize the solution $U(y)$ using a neural network and incorporate the PDE residual from (5) directly into the loss function. The network is trained to minimize a weighted combination of the PDE residual, boundary conditions, and additional regularization terms. Both their method and the approach used in this project rely on automatic differentiation for efficient evaluation of derivatives and gradients. However, while [2] uses neural networks as the ansatz space, this project employs a variation of the Chebyshev collocation method (cf. [1]), which can achieve spectral accuracy for smooth solutions, also using automatic differentiation for gradient-based optimization.

The remainder of this report is organized as follows. Section 2 introduces the localized Chebyshev collocation method used to discretize the boundary value problem (5) and presents numerical experiments that demonstrate the accuracy and efficiency of this spectral discretization for fixed, known values of λ. In Section 3, we turn to the more challenging problem of inferring the parameter λ for which the ODE (5) admits a smooth solution U, and we formulate and solve the resulting coupled optimization problem. Finally, Section 4 summarizes the key findings of the project and outlines furture directions.

2 Solving for fixed λ

2.1 Localized Chebyshev collocation

For values of λ that are not exactly equal to $\lambda = 1/(2(j+1))$ for $j = 0, 1, 2, \ldots$, the solution $U(y)$ to (5) exhibits nonsmooth behavior at the origin. For this project we specifically consider λ in a neighborhood of $\lambda = 0.5$, where the fourth derivative $U^{(4)}(y)$ becomes unbounded as $y \to 0$ for $\lambda \neq 0.5$. This nonsmoothness poses a challenge for standard spectral methods, which assume global smoothness of the solution. To overcome this, we utilize a localized Chebyshev collocation approach by dividing the domain $[-2, 2]$ into two subintervals: $[-2, 0]$ and $[0, 2]$, and discretize each segment with Chebyshev points, which naturally cluster near the boundaries of each segment. More concretely, for a given (even) resolution parameter n_y, we divide the domain $[-2, 2]$ into two subintervals, $[-2, 0]$ and $[0, 2]$. Each subinterval is discretized using $n_y/2+1$ Chebyshev collocation points, resulting in a total of $n_y + 1$ distinct points overall–with the endpoint $y = 0$ shared by both subintervals. For the remainder of this report, we denote

$$Y_1 = \{-2 = y_0, y_1, \ldots, y_{n_y/2} = 0\} \subset [-2, 0] \tag{7}$$

and

$$Y_2 = \{0 = y_{n_y/2}, y_{n_y/2+1}, \ldots, y_{n_y} = 2\} \subset [0, 2]. \tag{8}$$

to be the sets of Chebyshev collocation points in ascending order for the two subintervals, and subsequently, for a given profile U, we define

$$\mathbf{U}_1 = (U(y_0), \ldots, U(y_{n_y/2}))^t \in \mathbb{R}^{n_y/2+1} \quad \text{and} \quad \mathbf{U}_2 = (U(y_{n_y/2}), \ldots, U(y_{n_y}))^t \in \mathbb{R}^{n_y/2+1}. \tag{9}$$

The corresponding derivative vectors $D\mathbf{U}_1 \in \mathbb{R}^{n_y/2+1}$ and $D\mathbf{U}_2 \in \mathbb{R}^{n_y/2+1}$ can then be computed via spectral differentiation, and, further, higher order derivatives can be obtained analogously through repeated application of the derivative operator.

Remark 1. For notation clarity, the rest of the report uses zero indexing for vectors, e.g. $\mathbf{U}_1$ in (9) can equivalently be written as $\mathbf{U}_1 = ((\mathbf{U}_1)_0, (\mathbf{U}_1)_1, \ldots, (\mathbf{U}_1)_{n_y/2})^t \in \mathbb{R}^{n_y/2+1}$.

On each open segment $(-2, 0)$ and $(0, 2)$, the solution U remains smooth, allowing the spectral method to accurately capture its behavior within each piece. This clustering property is also intuitive: because $y = 0$ serves as the shared interface between the two subintervals, the Chebyshev points naturally accumulate near this nonsmooth region. As a result, we achieve enhanced resolution precisely where the solution demands finer discretization, allowing for more accurate representation and treatment of the challenging behavior near the origin. The advantage of this approach is illustrated in Figure 1, which compares the fourth derivative computed using three different methods: regular Chebyshev collocation on the full domain, localized Chebyshev collocation, and first-order finite differences. The plot clearly demonstrates that the localized method provides superior accuracy near the origin, where the fourth derivative becomes unbounded. The regular Chebyshev method struggles to capture this behavior due to its global smoothness assumption, while the localized approach naturally clusters points around the nonsmooth region, providing the necessary resolution to accurately represent the solution structure.

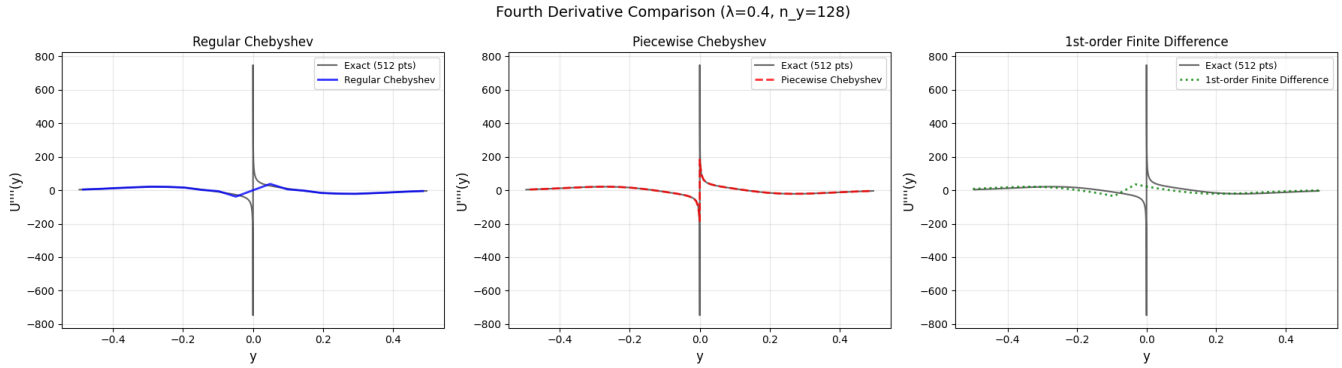

Figure 1: Comparison of fourth derivative $U^{(4)}(y)$ computed using different differentiation methods for $\lambda = 0.4$ and $n_y = 128$ discretization points. The exact solution (black line) is computed using spectral differentiation with 512 points. The plot demonstrates that the localized Chebyshev method provides superior accuracy near the origin, where the fourth derivative becomes unbounded.

2.2 Objective function and optimization

Rather than minimizing a weighted loss function that combines PDE residuals and boundary conditions as in the PINNS setting introduced in [2], we formulate the problem as the root-finding of a certain vector function $\mathcal{L}_\lambda : \mathbb{R}^{n_y/2+1} \times \mathbb{R}^{n_y/2+1} \to \mathbb{R}^{n_y+2}$ where n_y is the resolution parameter introduced in the preceding section. In detail, for a given λ, the vector function $\mathcal{L}_\lambda(\mathbf{U}_1, \mathbf{U}_2)$ is given by

$$\mathcal{L}_\lambda(\mathbf{U}_1, \mathbf{U}_2) = \begin{pmatrix} \mathcal{L}_\lambda^{\text{PDE}}(\mathbf{U}_1) \\ \mathcal{L}_\lambda^{\text{PDE}}(\mathbf{U}_2) \\ \mathcal{L}_\lambda^{\text{BC}}(\mathbf{U}_1, \mathbf{U}_2) \\ \mathcal{L}_\lambda^{\text{interface}}(\mathbf{U}_1, \mathbf{U}_2) \end{pmatrix} \tag{10}$$

where

$$(\mathcal{L}_\lambda^{\text{PDE}}(\mathbf{U}))_{i-1} = -\lambda(\mathbf{U})_i + ((1+\lambda)y_i + (\mathbf{U})_i)(D\mathbf{U})_i, \quad i = 1, 2, \ldots, n_y/2 - 1, \tag{11}$$

$$(\mathcal{L}_\lambda^{\mathrm{BC}}(\mathbf{U}_1, \mathbf{U}_2))_i = \begin{cases} (\mathbf{U}_1)_0 - 1, & i = 0, \\ (\mathbf{U}_2)_{n_y/2} - 1, & i = 1, \end{cases} \tag{12}$$

and

$$(\mathcal{L}_\lambda^{\mathrm{interface}}(\mathbf{U}_1, \mathbf{U}_2))_i = \begin{cases} (\mathbf{U}_1)_{n_y/2} - (\mathbf{U}_2)_0, & i = 0, \\ (D\mathbf{U}_1)_{n_y/2} - (D\mathbf{U}_2)_0, & i = 1. \end{cases} \tag{13}$$

These residuals enforce the boundary value problem within each subinterval, while simultaneously imposing continuity and flux matching conditions at the interface $y = 0$. This ensures both the solution and its first derivative are properly matched at the subdomain boundary, helping capture the coupling between the two regions.

To solve this system, we employ Newton-type methods (specifically, Powell's hybrid method) with the Jacobian computed exactly using automatic differentiation via JAX. Because convergence to the correct solution becomes more delicate at higher resolutions (larger n_y), we employ a multigrid-style strategy for constructing initial guesses. We first solve the system on a coarse grid, interpolate the solution onto a finer grid using piecewise linear interpolation, and use this interpolated solution as the initial guess for the solver at the increased resolution. Iterating this process from coarse to fine grids provides high-quality initial guesses, greatly improving convergence and robustness of the solver.

2.3 Fixed λ: numerical results

In this section, we present the effectiveness of the proposed method for solving the boundary value problem (5) for a fixed λ. We consider the non-smooth case where $\lambda = 0.4$ and the smooth case where $\lambda = 0.5$, using a discretization of $n_y = 64$ for the numerical solution. Figure 2 displays the numerical solution $U(y)$ and its first four derivatives $U'(y)$, $U''(y)$, $U'''(y)$, and $U^{(4)}(y)$ for both values of λ. The numerical solutions (blue solid lines) are obtained using the localized Chebyshev collocation method described in Section 2.1 and the root-finding approach with Powell's hybrid method from Section 2.2, where the Jacobian is computed using automatic differentiation. The exact solutions (red dashed lines, see eq. (4)) serve as reference. The derivative of the exact solution are computed via the spectral differentiation method with the localized Chebyshev nodes described in Seciton 2.1 using the finer resolution $n_y = 128$. In view of the rapid convergence of spectral differentiation for smooth functions, the approximation error becomes negligible relative to the error for the coarser resolution $n_y = 64$. Finally, we optimize to a tolerance of 10^{-9} and use multigrid levels $n_y = 2, 4, 8, 16$, and 32 for constructing initial guesses via piecewise linear interpolation.

Remark 2. We found that choosing $n_y = 64$ consistently produced the most accurate results across the first four derivatives. Smaller values of n_y were unable to adequately resolve the solution, while larger values led to differentiation matrices with prohibitively large condition numbers. For these reasons, all numerical experiments in this report use the choice $n_y = 64$.

In both cases, we observe that the numerical solution is in good agreement with the exact solution. For the case $\lambda = 0.4$, we observe an absolute ℓ_∞ error of 2.1×10^{-7} and a relative ℓ_2 error of 1.2×10^{-7}, while for the case $\lambda = 0.5$, we observe an absolute ℓ_∞ error of 1.7×10^{-7} and a relative ℓ_2 error of 1.1×10^{-7}. The solution $U(y)$ itself is smooth and well-captured, with the numerical method accurately reproducing the odd symmetry and boundary conditions $U(-2) = 1$ and $U(2) = -1$. The interface conditions at $y = 0$—requiring continuity of both the solution and its

first derivative—are also successfully recovered through the system formulation, as evidenced by the smooth transition visible in the plots. Although the numerical higher order derivatives (particularly the third and fourth) show instabilities near the boundaries, the method nevertheless resolves the key features distinguishing the two solutions: the cusp at the origin when $\lambda = 0.4$ and the smooth profile for $\lambda = 0.5$, achieving accuracies comparable to the PINN-based method reported in [2]. Further, the multigrid strategy employed for constructing initial guesses demonstrates efficient convergence behavior as shown in Table 1, typically requiring between 4 and 35 Newton iterations per level. Notably, the iteration count does not grow dramatically with increasing resolution, indicating that the piecewise linear interpolation between grid levels provides high-quality initial guesses that significantly accelerate convergence. We also note that in general, the smooth case ($\lambda = 0.5$) generally requires fewer iterations at intermediate levels compared to the non-smooth case ($\lambda = 0.4$).

Overall, the results demonstrate that the localized Chebyshev collocation method, combined with gradient-based optimization, provides an effective and accurate approach for solving the boundary value problem (5), even in the presence of non-smooth behavior at the origin. The method successfully handles both smooth and non-smooth cases, with the natural clustering of Chebyshev points near the origin ensuring adequate resolution where it is most needed.

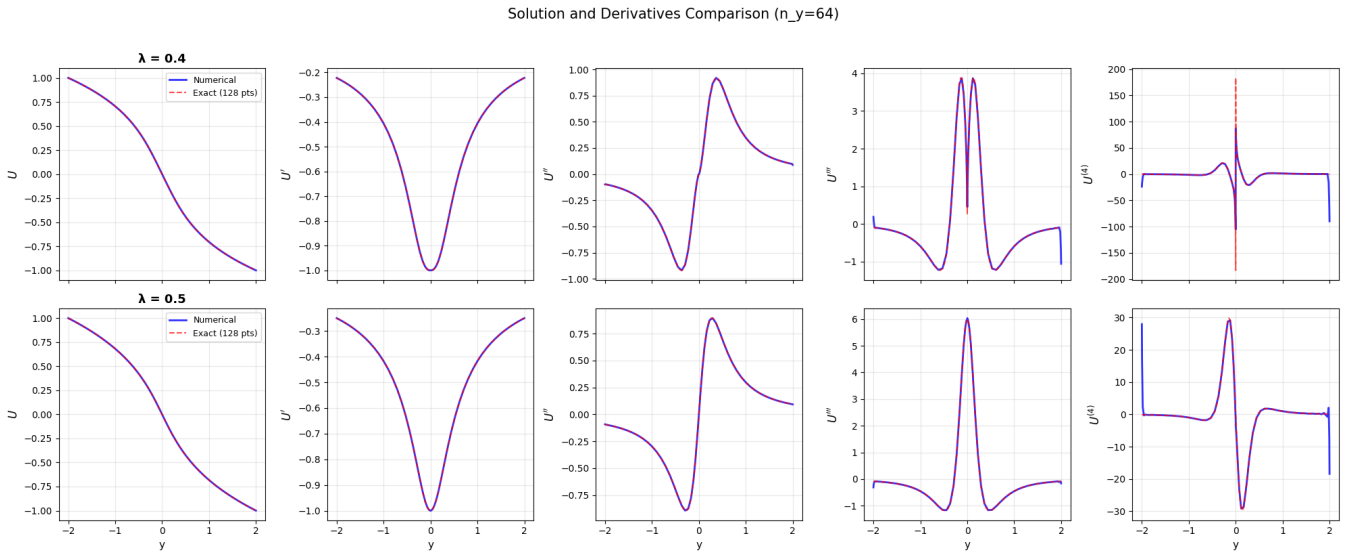

Figure 2: Comparison of numerical solutions (blue solid lines, $n_y = 64$) and exact solutions (red dashed lines) for the boundary value problem (5) with $\lambda = 0.4$ (top row) and $\lambda = 0.5$ (bottom row). Each row displays the solution $U(y)$ and its first four derivatives. The numerical method accurately captures both the smooth case ($\lambda = 0.5$) and the non-smooth case ($\lambda = 0.4$), where the fourth derivative becomes unbounded near the origin.

3 Inferring λ

3.1 Objective function and optimization

As mentioned in the preceding sections, our goal is to single out the smooth solution at $\lambda = 0.5$ from nearby non-smooth solutions by exploiting the fact that the non-smooth solutions have

n_y	Iterations ($\lambda = 0.4$)	Iterations ($\lambda = 0.5$)
2	5	5
4	14	4
8	20	22
16	20	22
32	35	30
64	26	18

Table 1: Newton iteration counts for multigrid levels considered in Section 2.3. We can see the multigrid strategy is effective in providing high-quality initial guesses, as iteration counts do not grow dramatically with increasing resolution.

discontinuous fourth derivative at the origin. In the PINN framework in [2], the optimization is performed simultaneously over both the parameter λ and the solution $U(y)$, where the solution is represented implicitly by neural network weights and a regularization term is added to penalize the fourth derivative of the neural network at points near the origin. We attempted to adopt the same strategy with out collocation-based approach, but we found that the joint optimization often failed to converge. We believe this difficulty reflects a fundamental difference in the two formulations. In the PINN setting, the optimization variables are network weights–parameters that do not directly corresond to pointwise values of the solution. The neural network itself provides a form of regularization, thereby improving conditioning for the problem. On the other hand, our approach optimizes directly over the solution values at the Chebyshev collocation points, rather than over network weights, introducing severe ill-conditioning when attempting to jointly optimize λ and U together. The fundamental issue arises from the fact that the fourth derivative operator D^4 is extremely ill-conditioned, with a condition number that grows rapidly with the number of collocation points. If we include a regularization term that includes the operator D^4 in the loss function and optimize over the grids $\mathbf{U}_1$ and $\mathbf{U}_2$, the overall problem becomes extremely ill-conditioned, making gradient-based optimization methods (such as Newton-type methods) numerically unstable and prone to convergence failures. This behavior is also intuitive: even very small perturbations in discrete solution values can produce extremely large errors in high-order derivatives, making simultaneous optimization over λ and U extremely difficult.

To address this difficulty, we adopt a nested optimization strategy combined with a gradient-free outer solver. For each fixed value of λ, let

$$\mathbf{U}^\lambda = \begin{pmatrix} \mathbf{U}_1^\lambda \\ \mathbf{U}_2^\lambda \end{pmatrix} \tag{14}$$

denote the solution of the fixed-λ problem described in Section 2.2. We then define the fourth-derivative interface scalar objective

$$\mathcal{L}^{\text{reg}}(\lambda) = \left| (D^4 \mathbf{U}_1^\lambda)_{n_y/2} - (D^4 \mathbf{U}_2^\lambda)_0 \right|, \tag{15}$$

and optimize this one-dimensional function using the Nelder–Mead method, a derivative-free algorithm. This decoupled approach avoids the severe ill-conditioning encountered in the joint optimization as the inner problem (solving for U at fixed λ) remains well behaved because it does not incorporate the fourth-derivative operator D^4 into the optimization itself, while the outer optimization introduces D^4 only at the λ level and does not involve optimization over the solution values.

To validate this approach, we plot the scalar objective $\mathcal{L}^{\text{reg}}(\lambda)$ over the interval $[0.45, 0.55]$ in Figure 3. Although $\mathbf{U}^\lambda$ contains numerical errors, the loss nevertheless attains a clear global minimum near the true value $\lambda = 0.5$, demonstrating the effectiveness of the scalarized nested optimization for estimating the optimal parameter. We also note that the loss is not convex, so in practice we initialize the Nelder–Mead algorithm from several random starting points within the interval and retain the solution corresponding to the smallest loss.

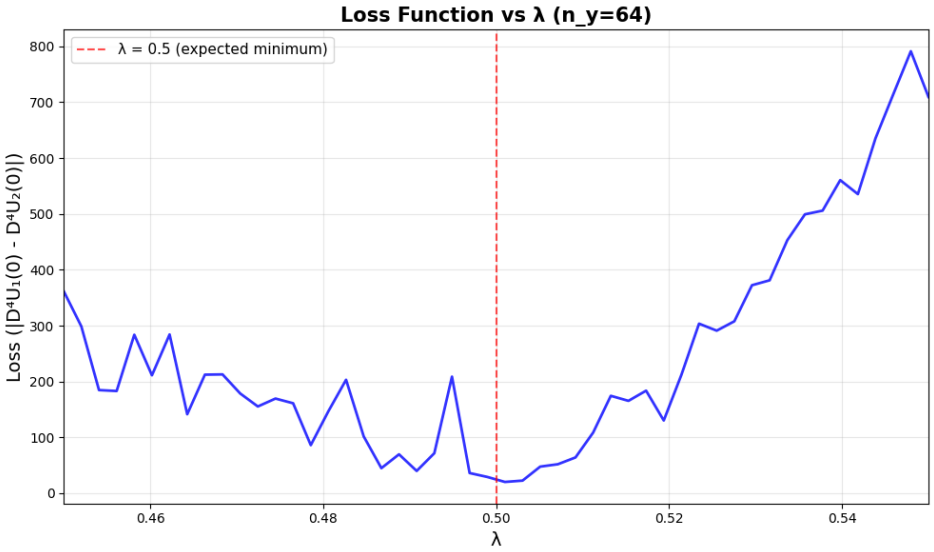

Figure 3: Scalar objective $\mathcal{L}^{\text{reg}}(\lambda)$ (see eq. (15)) as a function of λ. The sharp dip at $\lambda \approx 0.5$ indicates the location where the solution becomes smooth and the fourth-derivative jump vanishes.

3.2 Inferring λ: numerical results

In this section, we present the results of applying the nested optimization strategy to infer the parameter λ. As in Section 2.3, we use $n_y = 64$ collocation points (see Remark 2). We initialize the Nelder–Mead outer optimization from 20 random values of λ sampled uniformly from the interval $[0.45, 0.55]$. Table 2 summarizes the results of a single run. We can see the Nelder–Mead optimization terminated successfully for all 20 initial values, requiring between 29 and 34 iterations and between 72 and 88 loss evaluations, which demonstrates the tractability of this formulation. Further, the recovered parameter value, $\lambda = 0.49915829$, lies within 8.4×10^{-4} of the true smooth-solution value $\lambda = 0.5$. These results highlight the effectiveness of the approach: despite the markedly non-convex loss landscape—evidenced by the broad range of final loss values (from approximately 0.07 to 43)—the Nelder–Mead method is able to identify the global minimizer.

The primary drawback of this nested optimization framework is its computational cost. As shown in Table 2, each run required roughly 80 loss evaluations, resulting in a total of about $20 \times 80 = 1600$ evaluations. Thus, the nested strategy is approximately 1600 times more expensive than the fixed-λ optimization described in Section 2.3.

Metric	Value
Number of Initial Values	20
Successful convergences	20
Final λ value	0.49915829
Error from true value ($\lambda = 0.5$)	8.4×10^{-4}
Iterations: min/max/mean	29 / 34 / 31.0
Loss evaluations: min/max/mean	72 / 88 / 79.3
Final loss: min/max/mean	0.065 / 42.813 / 7.2

Table 2: Summary of Nelder–Mead optimization results from 20 random initializations in $[0.45, 0.55]$. Single run of Nelder–Mead optimization (see Section 3.1). The wide range of final function values (0.065 to 42.813) reflects the non-convex nature of the loss landscape, yet we successfully recovered the global minimum near $\lambda = 0.5$.

4 Conclusions

In this report, we developed a collocation-based framework for solving the self-similar Burgers equation and for inferring the parameter λ that distinguishes smooth from non-smooth similarity profiles. The localized Chebyshev method proved effective on each subinterval, accurately capturing both smooth and cusp-like behavior near the origin. Combined with automatic differentiation and a multigrid-style initialization strategy, this approach yielded accurate solutions for fixed λ. Because direct joint optimization over (λ, U) is severely ill-conditioned–due largely to the instability of high-order differentiation matrices–we adopted a nested strategy in which the fixed λ problem is solved in each iteration of the outer optimization, and λ is then inferred by minimizing a scalar measure of the fourth-derivative jump. Despite the non-convexity of this objective, the derivative-free Nelder–Mead method reliably recovered the correct smoothing parameter $\lambda = 0.5$ with several digits of accuracy. Overall, these results show that localized spectral discretization combined with nested optimization offers a robust framework for studying self-similar blowup profiles and identifying smoothness-determining parameters.

Future directions include extending the method to other discrete values of λ that admit smooth similarity solutions (including unstable blowups), examining regimes that require detecting nonsmoothness in even higher derivatives, and applying the approach to more complex nonlinear PDEs.

Remark 3 (Code)**.** All code used for the results in this report can be found at `https://github.com/ayang923/AM205-project`.

References

[1] T. A. Driscoll and N. Hale. Rectangular spectral collocation. *IMA Journal of Numerical Analysis*, 36(1):108–132, 2016.

[2] Y. Wang, C.-Y. Lai, J. Gómez-Serrano, and T. Buckmaster. Asymptotic self-similar blowup profile for three-dimensional axisymmetric euler equations using neural networks. *Physical Review Letters*, 130(24):244002, 2023.